

Manual de Uso do GitHub — Sincronização, Branches e Rollback

Icatu Brasil — GIB

Objetivo: cada pessoa do time faz respaldo periódico do próprio trabalho em uma branch, testa, e só depois junta (merge) com a `main`.

1. Como acessar ou trocar de organização (perfil)

O GitHub permite alternar entre a sua conta pessoal e organizações das quais você faz parte (ex.: GrupoIcatuBrasil).

1. Clicar no seu nome de usuário/avatar no canto superior esquerdo
 2. Na janela “Go to organization dashboard”, escolher a organização desejada (ex.: GrupoIcatuBrasil)
 3. Isso muda o contexto: repositórios, criação de repositório e demais ações passam a ser feitas dentro dessa organização
-

2. Como criar um repositório

No GitHub (repositório remoto)

1. Acessar github.com → clicar em **New repository**
2. Definir um nome para o repositório
3. Escolher **Private** ou **Public**
4. Clicar em **Create repository**

No computador (repositório local)

Dentro da pasta do projeto, no Git Bash:

```
git init
```

3. Como conectar o projeto do computador com o repositório do GitHub

```
git remote add origin https://github.com/usuario/nome-do-repositorio.git
git branch -M main
git push -u origin main
```

Para conferir se a conexão já existe:

```
git remote -v
```

4. Arquivos de configuração que podem ser incluídos no projeto

Arquivo	Para que serve
.gitignore	Lista pastas/arquivos que o Git deve ignorar e nunca subir (ex.: node_modules/, arquivos de senha, .env)
README.md	Descrição do projeto, mostrada na página principal do repositório no GitHub
.gitattributes	Define regras de tratamento de arquivos (ex.: quebra de linha, arquivos binários)

5. Agendar sincronização periódica (diária, semanal, etc.)

O Git em si **não tem um “agendador” nativo de push/pull**. Existem duas abordagens:

Opção A — Rotina manual (recomendada para o time)

Definir um horário fixo (ex.: final do expediente, ou toda sexta) e seguir sempre os mesmos 3 comandos no Git Bash:

```
git add .
git commit -m "descrição curta do que foi feito"
git push
```

Mais simples, mais seguro (você revisa antes de enviar) e não exige configuração extra.

Opção B — Automatizar um lembrete ou rotina no Windows

Dá para criar uma tarefa agendada no Agendador de Tarefas do Windows que rode um script .bat chamando os mesmos comandos acima, em horário fixo (diário/semanal).

Risco a apontar: automatizar o `git push` sem revisão humana pode enviar código quebrado ou incompleto para o repositório sem ninguém perceber. **Recomendação:** automatize no máximo o lembrete (ex.: notificação do Windows “hora de sincronizar”), mas mantenha o `commit/push` manual, revisado por quem está fazendo.

6. Sincronizar com uma branch nova ou existente

Criar e sincronizar uma branch nova

```
git checkout -b nome-da-branch      # cria e já muda para ela
git push -u origin nome-da-branch    # publica no GitHub e liga
local↔remoto
```

Depois da primeira vez, basta `git push` normalmente.

Sincronizar com uma branch já existente

```
git fetch origin          # busca atualizações sem aplicar
git checkout nome-da-branch # muda para a branch
git pull                  # traz o que tem de novo no GitHub
```

Padrão sugerido: uma branch por pessoa ou por tarefa (ex.: samuel-backup, feature-relatorio-x), nunca trabalhar direto na main.

7. Juntar (merge) a branch com a main

Caminho recomendado — Pull Request (PR)

Mais seguro porque permite revisão antes de afetar a main.

1. No GitHub, abrir o repositório → aba Pull requests → New pull request
2. Selecionar: base = main, compare = sua branch
3. Descrever o que foi alterado
4. Pedir revisão de alguém do time (ou revisar você mesmo antes de confirmar)
5. Clicar em Merge pull request só depois de validar (ver seção 5)

Caminho direto (linha de comando)

Usar apenas se for baixo risco e já validado:

```
git checkout main
git pull
git merge nome-da-branch
git push
```

Recomendação: para qualquer coisa que afete um sistema em produção, usar sempre PR — dá rastro de quem aprovou e quando.

8. Visibilidade das branches, responsáveis e timeline

O que você quer ver	Onde/como
Todas as branches existentes	GitHub → aba Branches (mostra quem é o “author” da última alteração)
Quem fez cada commit	GitHub → aba Commits, ou clicar em uma linha de código → Blame
Linha do tempo visual de todas as branches	GitHub → Insights → Network (gráfico com todos os ramos)
Pull Requests abertos/fechados e por quem	GitHub → aba Pull requests
Ver no terminal (Git Bash)	<code>git log --oneline --graph --all</code> (histórico visual de todas as branches locais conhecidas)

O que você quer ver	Onde/como
Ver branches e com quem estão sincronizadas	<code>git branch -vv</code>

9. Como validar e fazer rollback

Validar antes de aceitar o merge

Checklist mínimo antes de aprovar um PR: - Testar a branch localmente (git checkout nome-da-branch, rodar o sistema) - Revisar o “diff” (as mudanças) na própria tela do PR no GitHub - Confirmar que não há dado sensível (senha, CPF, chave de API) no código antes de subir

Rollback (desfazer)

Se o problema já foi identificado antes do merge: simplesmente não aprove o PR, ou continue ajustando a branch.

Se o problema foi identificado depois do merge na main:

Situação	Comando/ação	Efeito
Já foi para produção / outras pessoas já puxaram a main	<code>git revert</code> ou botão Revert no próprio PR no GitHub	Cria um novo commit que desfaz a mudança, sem apagar histórico — opção mais segura
Ainda local, ninguém mais puxou	<code>git reset --hard</code>	Reescreve o histórico — risco: se alguém já puxou essa main, gera conflito para o time inteiro. Evitar em branch compartilhada

Recomendação geral: em caso de dúvida, sempre usar revert, nunca reset --hard em branch compartilhada.